Computación y estructuras discretas I

Unidad 3: Estructuras Discretas no recursivas y análisis de algoritmos – Crecimiento de funciones y Análisis de algoritmos

Juan Marcos Caicedo Mejía – jmcaicedo@icesi.edu.co

Departamento de Computación y Sistemas Inteligentes
Facultad Barberi de Ingeniería, Diseño y Ciencias Aplicadas
Universidad ICESI
Material adaptado del material original del profesor Oscar Bedoya

2026

Tabla de contenido

1. Crecimiento de funciones
2. Notación O
3. Notación Ω
4. Notación Θ
5. Terminología de complejidades
6. Clasificación de problemas
7. Análisis de algoritmos

Contenido

1. Crecimiento de funciones

2. Notación O

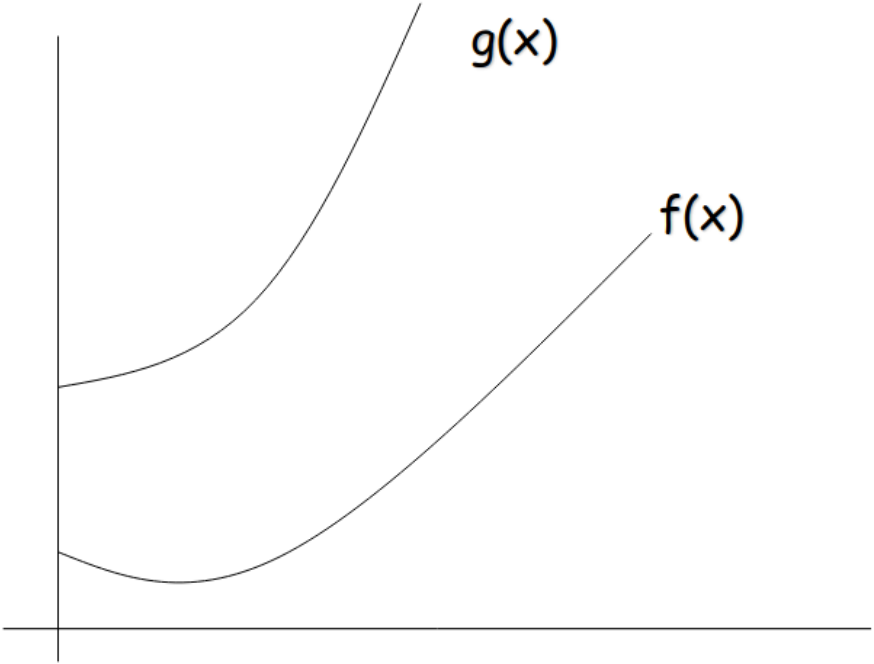
3. Notación Ω

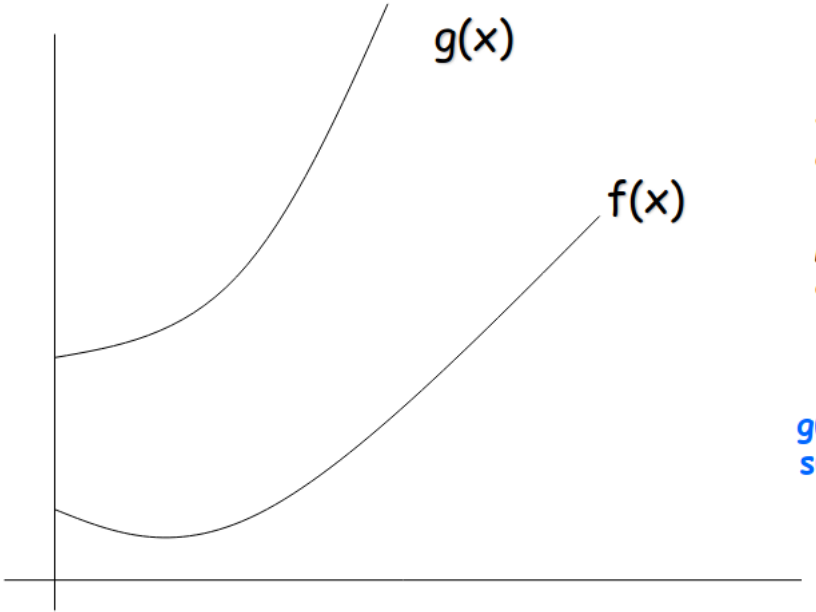
4. Notación Θ

5. Terminología de complejidades

6. Clasificación de problemas

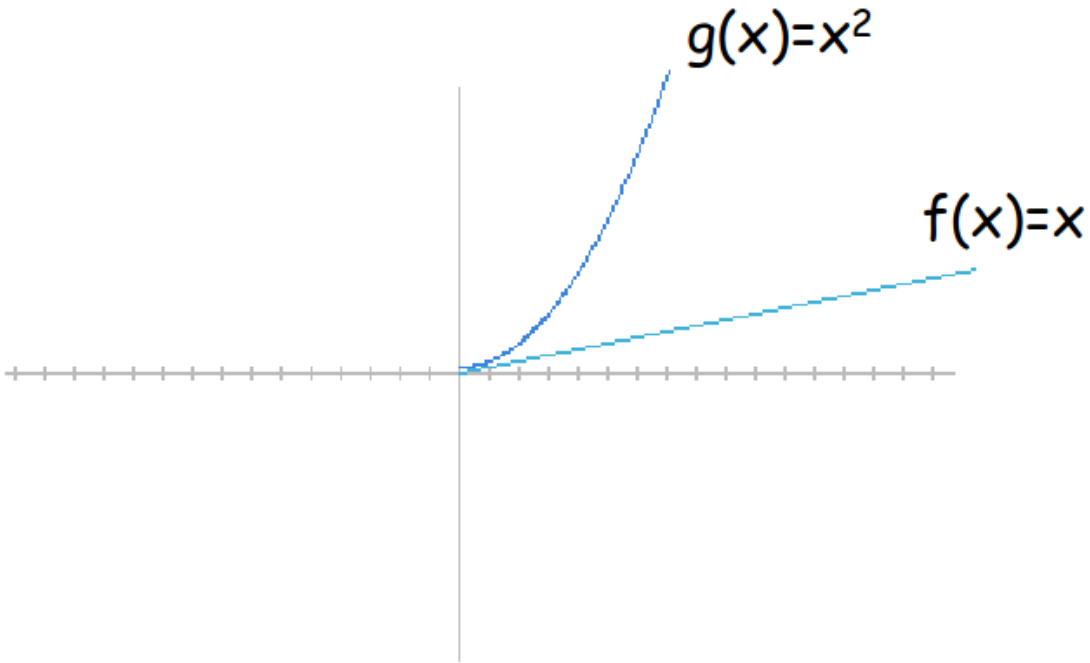
7. Análisis de algoritmos

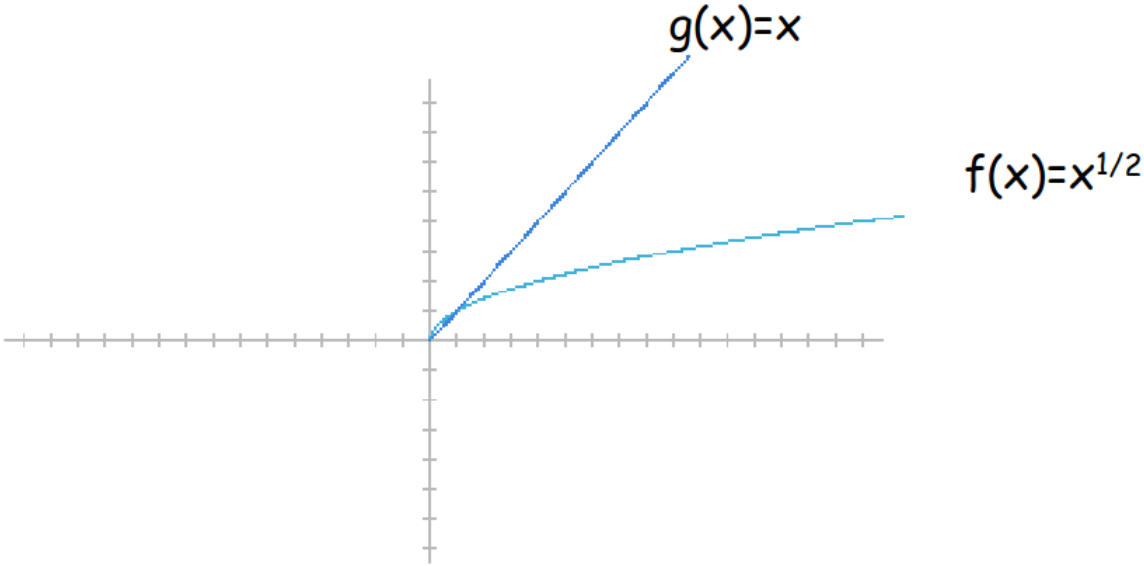

g(x)
f(x)

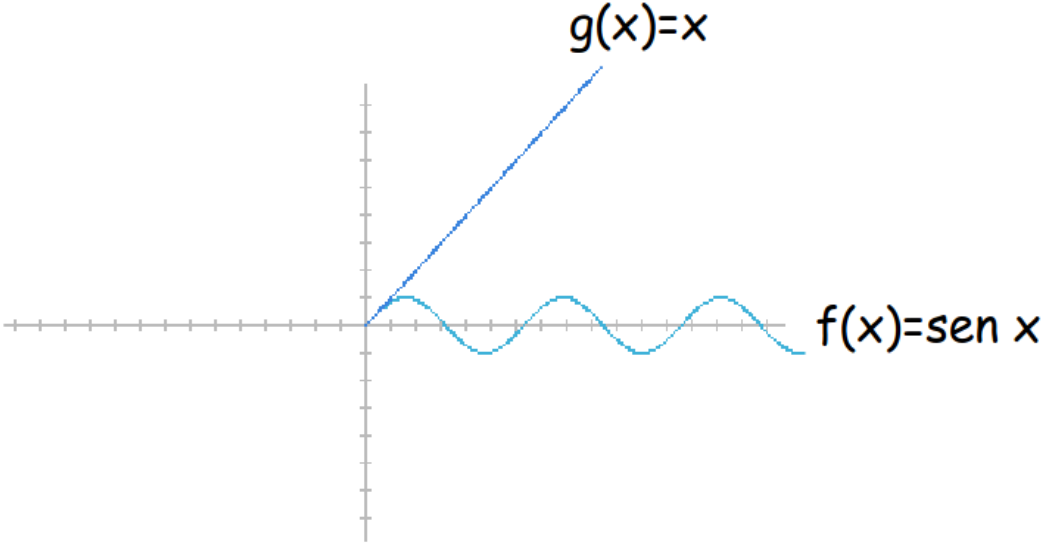


Siempre se cumple que f(x)<g(x) para cualquier valor x

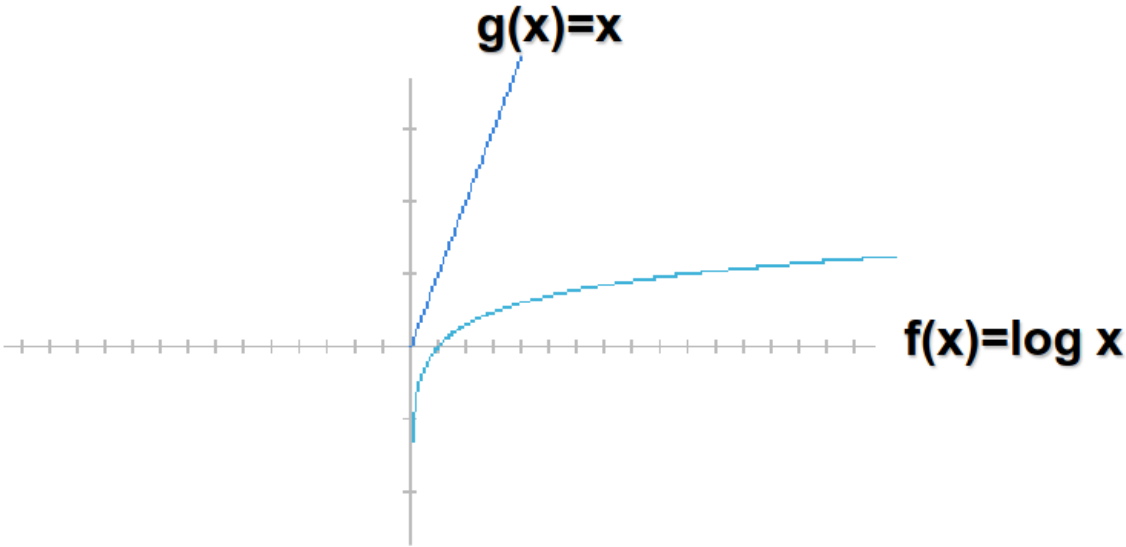
g(x) es una cota superior de f(x)

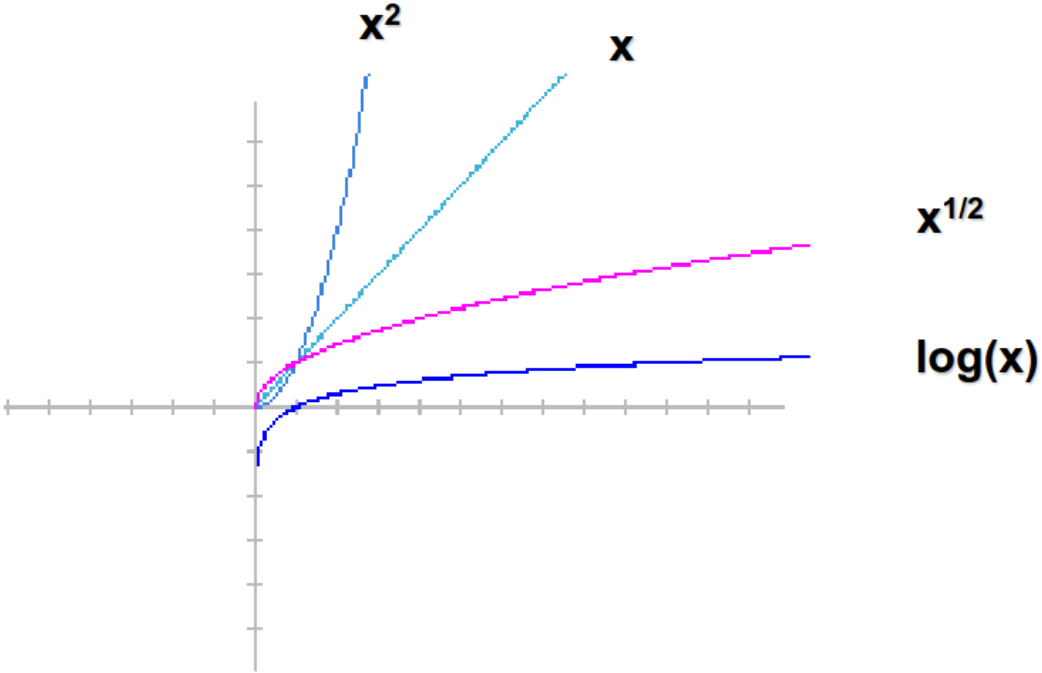

g(x)=x²
f(x)=x

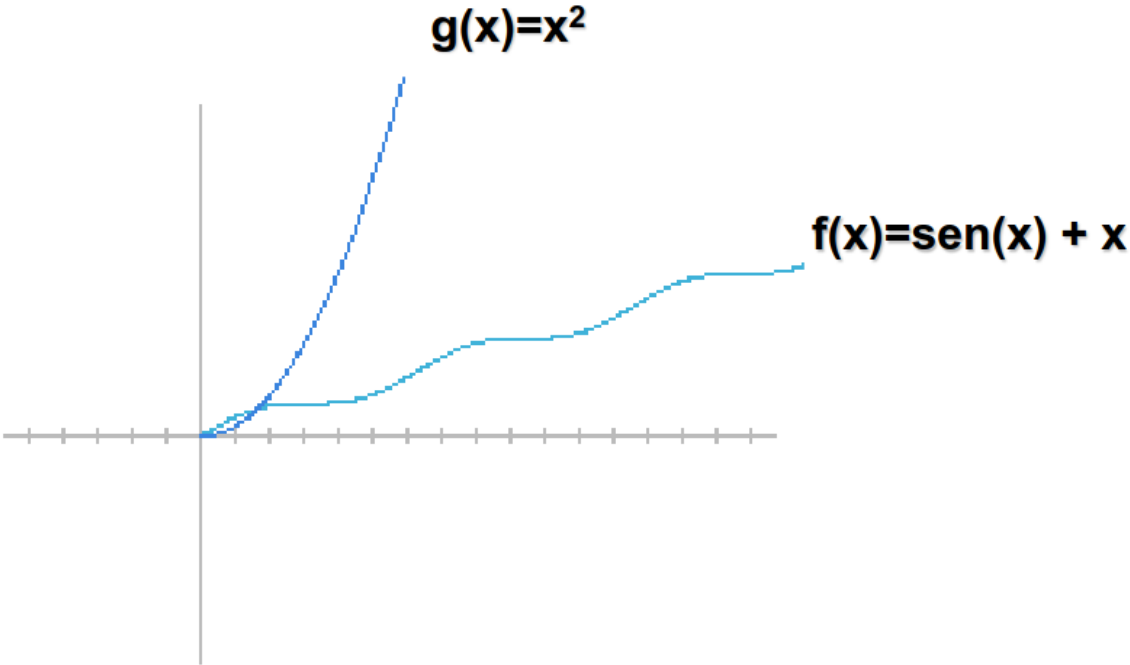

g(x)=x
f(x)=x^1/2



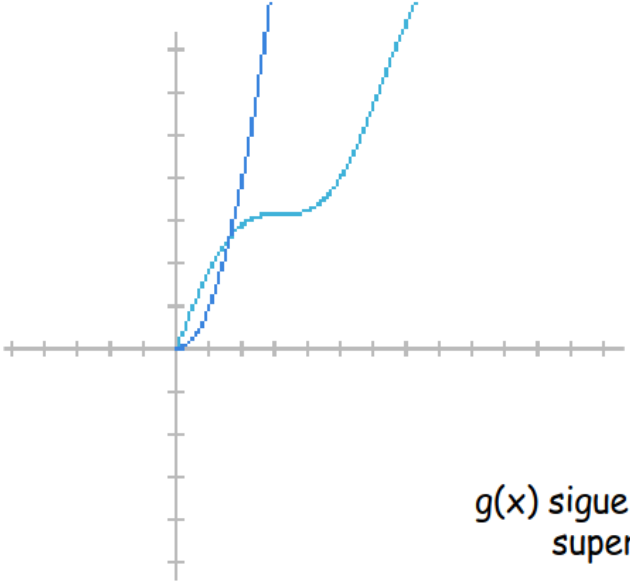
¿Está (sen x) por debajo de x^2?


g(x)=x
f(x)=log x


x^2
x
$x^{1/2}$
$\log(x)$

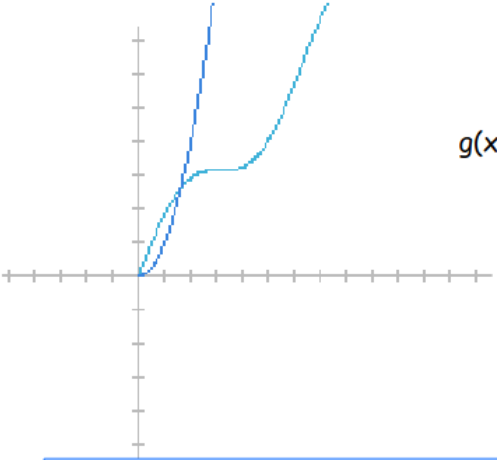

g(x)=x²
f(x)=sen(x) + x

$g(x)=x^2$ $f(x)=sen(x) + x$



g(x) sigue siendo una cota superior de f(x)

$g(x)=x^2$ $f(x)=sen(x) + x$

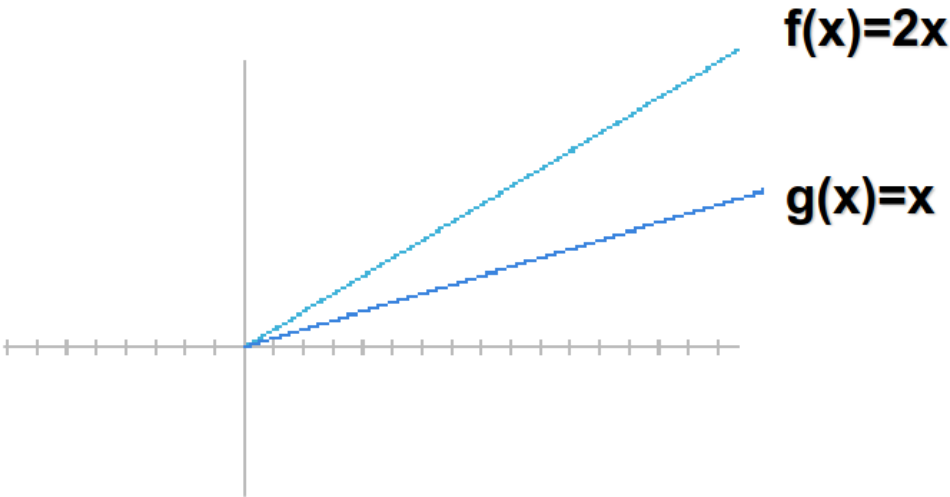


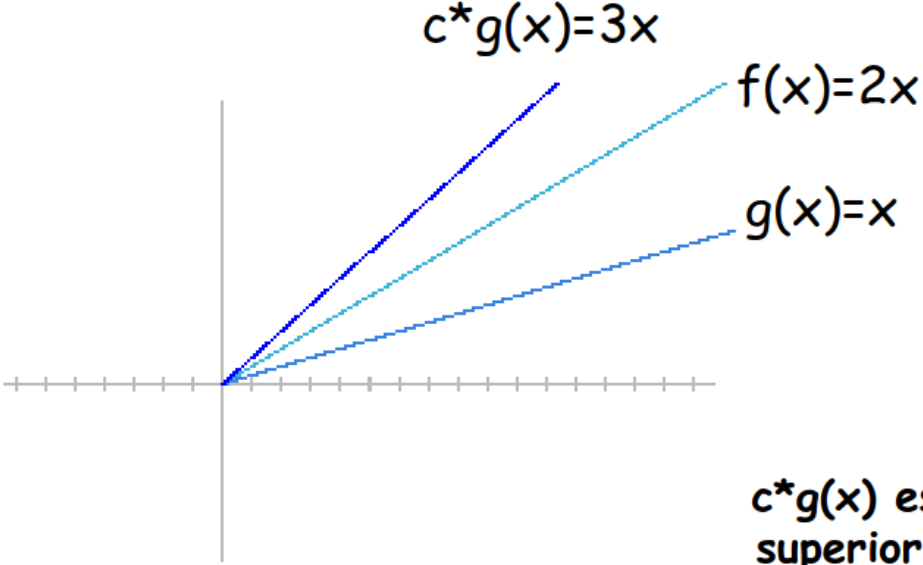
g(x) sigue siendo una cota superior de f(x)

Para que una función sea cota superior de otra, debe serlo para <u>todos</u> los valores de **x≥k**

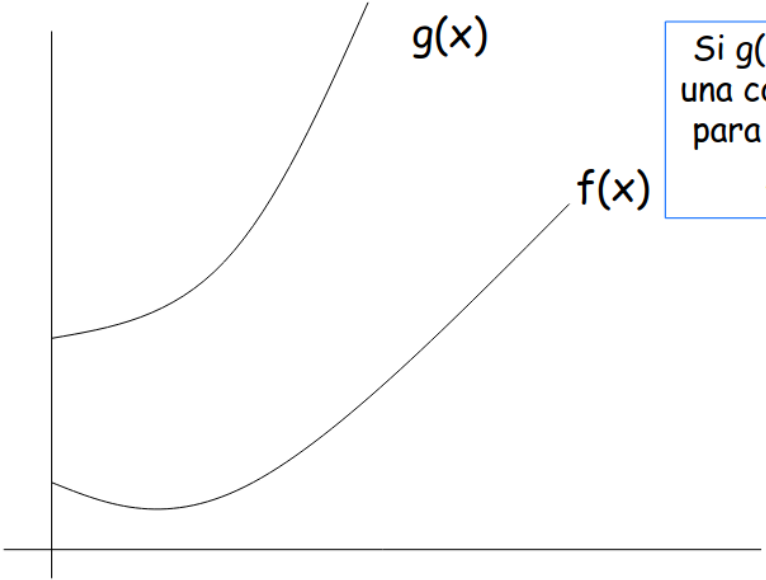
Pueden existir valores de x<k para los cuales no sea cota superior

Buscar una cota superior para f(x) en términos de g(x)



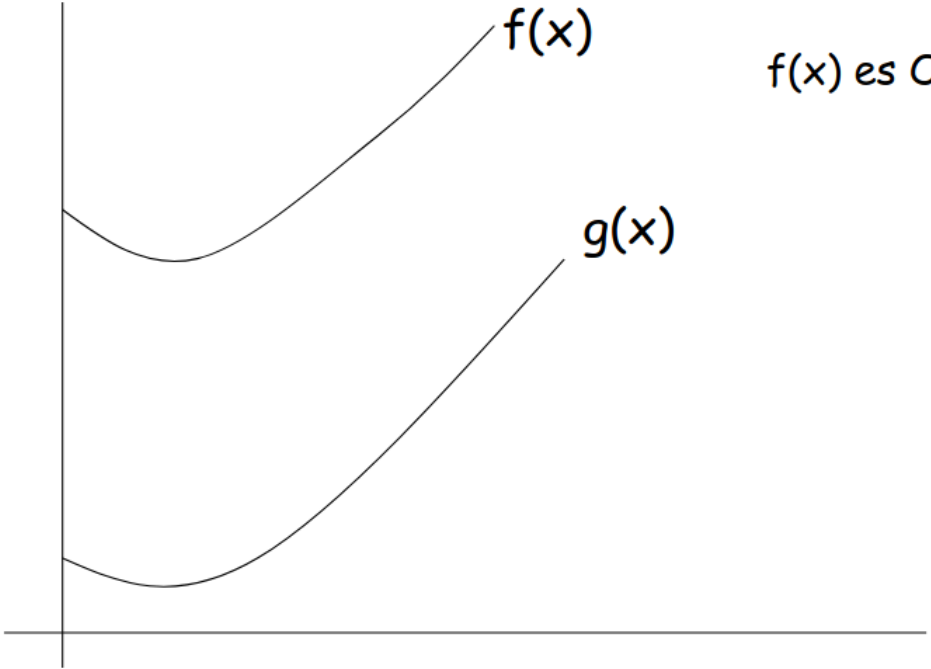


c*g(x) es una cota superior para f(x)

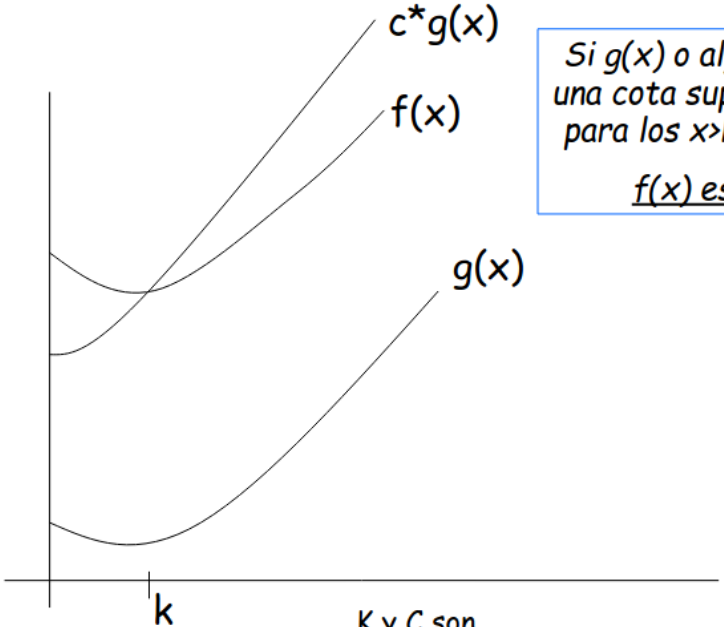


Si g(x) o algún c*g(x) es una cota superior de f(x), para los x>k, se dice que

f(x) es O(g(x))



f(x) es O(g(x)) ?



K y C son constantes

*Si g(x) o algún c*g(x) es una cota superior de f(x), para los x>k, se dice que*

<u>f(x) es O(g(x))</u>

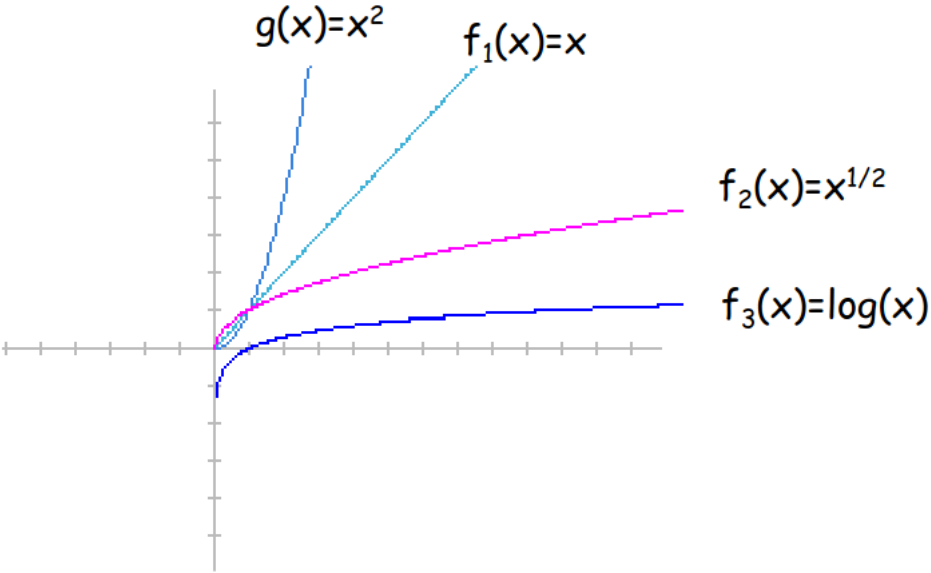
Crecimiento de funciones

Dadas dos funciones f y g, se dice que $f(x)$ es $O(g(x))$ si existen constantes c y k tales que:

$$f(x) \leq c \cdot g(x)$$

se cumple para todos los $x \geq k$

Ψ *Donald Knuth (1938 -) - Símbolo de Landau*



g(x) es una cota superior de
$f_1(x)$, $f_2(x)$, $f_3(x)$

Contenido

1. Crecimiento de funciones

2. Notación O

3. Notación Ω

4. Notación Θ

5. Terminología de complejidades

6. Clasificación de problemas

7. Análisis de algoritmos

Crecimiento de funciones

Formalmente la notación O representa un conjunto de funciones

$O(g(x)) = \{f(x) \mid \text{existen constantes positivas } c \text{ y } k, \text{ tales que } 0 \leq f(x) \leq c \cdot g(x), \text{ para todos los } x \geq k\}$

Crecimiento de funciones

$$O(x^2) = \{x, x^{1/2}, \log(x), \ldots\}$$

Crecimiento de funciones

Muestre que $7x^2 = O(x^3)$

Crecimiento de funciones

Muestre que $7x^2 = O(x^3)$

Exploración: Buscamos desde qué valor de x se cumple $7x^2 \leq x^3$

x	x^3	$7x^2$	
1	1	7	✗
2	8	28	✗
3	27	63	✗
4	64	112	✗
5	125	175	✗
6	216	252	✗
7	343	343	✓
8	512	448	✓

A partir de $x \geq 7$ se cumple:

$$7x^2 \leq x^3$$

Por lo tanto:

$$f(x) \leq 1 \cdot g(x)$$

para $x \geq 7$

$c = 1, \quad k = 7$

Crecimiento de funciones

Es x^3, $O(7x^2)$

Crecimiento de funciones

Es x^3, $O(7x^2)$

$x^3 < c\ 7x^2$

$x < 7c$

Ya que no se cumple para todos los $x > k$, no es cierto que x^3 sea $O(7x^2)$

Crecimiento de funciones

Ejercicios

Demostrar que:

- $x^2 + 2x + 1$ es $O(x^2)$
- $\log n$ es $O(n)$, parta sabiendo que $n < 2^n$
- $x^2 + 4x + 17$ es $O(x^3)$ pero que x^3 no es $O(x^2 + 4x + 17)$
- $x \log x$ es $O(x^2)$

Consulta

- Explique qué significa que una función sea $\Omega(1)$
- Explique qué significa que una función sea $O(1)$

Crecimiento de funciones

Cota superior asintótica

Cuando se determina que el tiempo de computo de un algoritmo es $O(g(x))$ se establece una cota superior

Dentro del análisis para obtener la cota superior, se debe considerar el peor caso, de esta forma se consigue tener una cota que no puede resultar peor

Crecimiento de funciones

Cota superior asintótica

Suponga que para un algoritmo se encontró que $T_1(n) = O(n^2)$ y para otro que $T_2(n) = O(n \lg n)$, qué representan estos resultados?

¿Qué se puede esperar en cuanto al tiempo de ejecución de los algoritmos?

¿Se puede asegurar que siempre es mejor el algoritmo 2 que el 1?

Crecimiento de funciones

Cota superior asintótica

Es un abuso decir que el tiempo de ejecución del insertion sort es $O(n^2)$, esto se puede asegurar, para el peor caso.

Contenido

1. Crecimiento de funciones

2. Notación O

3. Notación Ω

4. Notación Θ

5. Terminología de complejidades

6. Clasificación de problemas

7. Análisis de algoritmos

Crecimiento de funciones

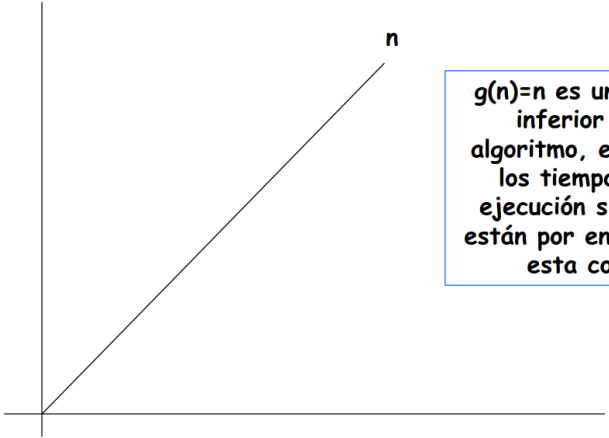
Cota inferior

$\Omega(g(x)) = \{\, f(x) \mid$ existen constantes positivas c y k, tales que $0 \leq c \cdot g(x) \leq f(x)$, para todos los $x \geq k \,\}$

Crecimiento de funciones

Cota inferior

Suponga que para un algoritmo se encontró que $T_1(n) = \Omega(n)$



g(n)=n es una cota inferior del algoritmo, esto es, los tiempos de ejecución siempre están por encima de esta cota

Crecimiento de funciones

Cota inferior

Suponga que para un algoritmo se encontró que $T_1(n) = \Omega(n^2)$ y para otro que $T_2(n) = \Omega(n \lg n)$, qué representan estos resultados?

¿Qué se puede esperar en cuanto al tiempo de ejecución de los algoritmos?

Contenido

1. Crecimiento de funciones

2. Notación O

3. Notación Ω

4. Notación Θ

5. Terminología de complejidades

6. Clasificación de problemas

7. Análisis de algoritmos

Crecimiento de funciones

Notación $\Theta(f(n))$

$f(n) = \Theta(g(n))$ sii $f(n) = O(g(n))$ y $f(n) = \Omega(g(n))$

Contenido

1. Crecimiento de funciones

2. Notación O

3. Notación Ω

4. Notación Θ

5. Terminología de complejidades

6. Clasificación de problemas

7. Análisis de algoritmos

Crecimiento de funciones

Complejidades de los algoritmos

Complejidad	Terminología
$O(1)$	Complejidad constante
$O(\log n)$	Complejidad logarítmica
$O(n)$	Complejidad lineal
$O(n \log n)$	Complejidad $n \log n$
$O(n^b)$	Complejidad polinomial
$O(b^n)$	Complejidad exponencial
$O(n!)$	Complejidad factorial

Contenido

1. Crecimiento de funciones

2. Notación O

3. Notación Ω

4. Notación Θ

5. Terminología de complejidades

6. Clasificación de problemas

7. Análisis de algoritmos

Crecimiento de funciones

Complejidades de los algoritmos

Un problema que se puede resolver utilizando un algoritmo con complejidad polinómica en el peor caso se llama tratable. De no se así se llama intratable

Un problema para el cual no existe un programa que lo pueda solucionar se llama irresoluble. De no ser así se llaman resolubles. (Halting problem - Turing)

Ψ Alan Turing (1912–1954) Problema de la decisión de Hilbert

Crecimiento de funciones

Complejidades de los algoritmos

Existen problemas que no se pueden resolver, en el peor caso en tiempo polinomial, pero que dada alguna solución, se puede comprobar que es efectivamente una solución. Estos problemas se llaman NP (Polinómico - No determinista)

Existe un conjunto de problemas NP que además, si se llegase a encontrar una solución en tiempo polinómico, daría solución a un conjunto de problemas relacionados. Este tipo de problema se conoce como NP-completos

Complejidades de los algoritmos

El problema de la satisfactibilidad es un problema NP-completo.

Dada una asignación de valores de verdad, se puede verificar en tiempo polinómico si tal asignación satisface, o no, una fórmula proposicional. Sin embargo, no existe un algoritmo que en tiempo polinómico pueda encontrar la asignación de valores de verdad para que una fórmula cualquiera se satisfaga.

Contenido

1. Crecimiento de funciones

2. Notación O

3. Notación Ω

4. Notación Θ

5. Terminología de complejidades

6. Clasificación de problemas

7. Análisis de algoritmos

Análisis de algoritmos

#	Instrucción	Costo
1	$i \leftarrow 1$	c_1
2	**while** $i \leq$ len(mat1)	c_2
3	$\quad j \leftarrow 1$	c_3
4	$\quad$ **while** $j \leq$ len(mat2)	c_4
5	$\quad\quad mat3[i][j] \leftarrow mat1[i][j] + mat2[i][j]$	c_5
6	$\quad\quad j \leftarrow j + 1$	c_6
7	$i \leftarrow i + 1$	c_7